



DA-RAG: Dynamic Attributed Community Search for Retrieval-Augmented Generation

Xingyuan Zeng
zengxy96@mail2.sysu.edu.cn
The Technology Innovation Center
for Collaborative Applications of
Natural Resources Data in GBA, MNR
Sun Yat-sen University
Zhuhai, China

Zuohan Wu
zh.wu@connect.hkust-gz.edu.cn
The Hong Kong University of Science
and Technology (Guangzhou)
Guangzhou, China

Yue Wang
yuewang@sics.ac.cn
Shenzhen Institute of Computing
Sciences
Shenzhen, China

Chen Zhang
jason-c.zhang@polyu.edu.hk
The Hong Kong Polytechnic
University
Hong Kong, China

Quanming Yao
qyaoaa@tsinghua.edu.cn
Tsinghua University
State Key laboratory of Space
Network and Communications
Beijing National Research Center for
Information Science and Technology
Beijing, China

Libin Zheng*
Jian Yin
zhenglb6@mail.sysu.edu.cn
issjyin@mail.sysu.edu.cn
Sun Yat-sen University
Zhuhai, China

Abstract

Owing to their unprecedented comprehension capabilities, large language models (LLMs) have become indispensable components of modern web search engines. From a technical perspective, this integration represents retrieval-augmented generation (RAG), which enhances LLMs by grounding them in external knowledge base. A prevalent technical approach in this context is graph-based RAG (G-RAG). However, current G-RAG methodologies frequently underutilize graph topology, predominantly focusing on low-order structures or pre-computed static communities. This limitation affects their effectiveness in addressing dynamic and complex queries. Thus, we propose DA-RAG, which leverages attributed community search (ACS) to dynamically extract relevant subgraphs based on the queried question. DA-RAG captures high-order graph structures, allowing for the retrieval of self-complementary knowledge. Furthermore, DA-RAG is equipped with a chunk-layer oriented graph index, which facilitates efficient multi-granularity retrieval while significantly reducing both computational and economic costs. We evaluate DA-RAG on multiple datasets, demonstrating that it outperforms existing RAG methods by up to 40% in head-to-head comparisons across four metrics while reducing index construction time and token overhead by up to 37% and 41%, respectively.

CCS Concepts

• **Information systems** → **Question answering**.

*Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License. *WWW '26, Dubai, United Arab Emirates*.
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792430>

Keywords

Graph-based Retrieval-Augmented Generation; Attributed Community Search; Graph Mining

ACM Reference Format:

Xingyuan Zeng, Zuohan Wu, Yue Wang, Chen Zhang, Quanming Yao, Libin Zheng, and Jian Yin. 2026. DA-RAG: Dynamic Attributed Community Search for Retrieval-Augmented Generation. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3774904.3792430>

Resource Availability:

The source code of this paper has been made publicly available at <https://doi.org/10.5281/zenodo.18296495>.

1 Introduction

Retrieval-Augmented Generation (RAG) [1, 2] has emerged as a prominent technique for enhancing large language models (LLMs). An exemplary RAG application is Microsoft Copilot¹, which once sparked a trend of integrating LLMs into web search. On the one hand, for search engines, LLMs can summarize the desired content for users as an indispensable assistant in the modern industry [3]. On the other hand, by incorporating relevant context retrieved from external knowledge sources, RAG enables LLMs to generate accurate, timely, and domain-specific responses without altering their underlying parameters. Among the various emerging RAG paradigms, graph-based RAG (G-RAG) [4–6] is gaining popularity, due to its advantage of employing a graph to represent the relationships among data entities. Compared to traditional RAG approaches [1, 2, 7], which treat documents or text chunks as discrete and independent units, G-RAG captures the complex semantic relationships within the data. This helps to retrieve a knowledge collection that is inherently correlative and complementary.

¹<https://copilot.microsoft.com/>

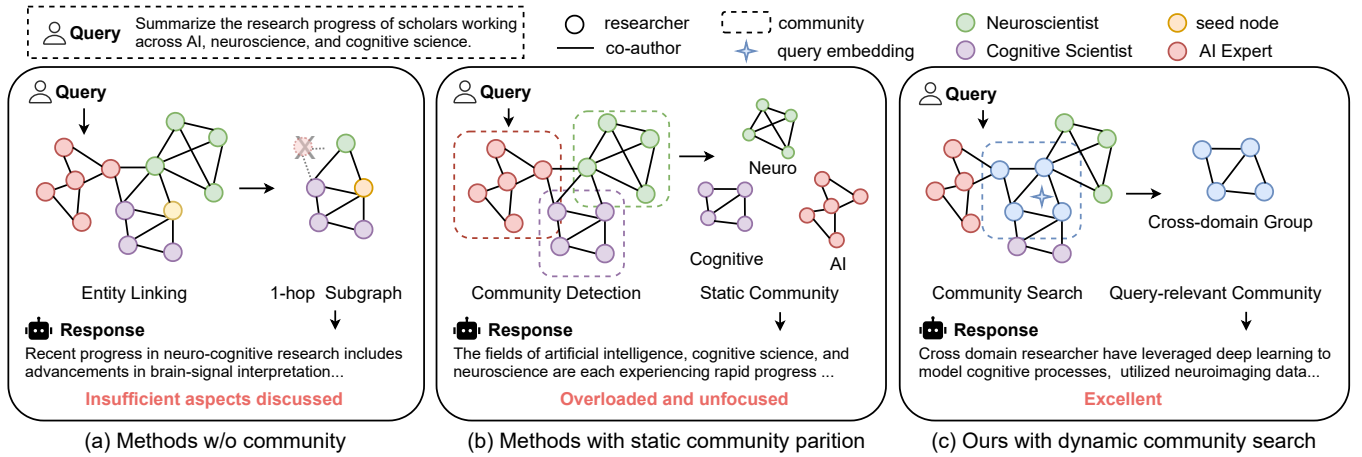


Figure 1: Differences between existing methods and our method. (a) Methods w/o community concern are limited to low-order graph topology, capturing only partial aspects. (b) Methods with static community partition could return a diverging and unfocused response. (c) Our method retrieves a query-relevant subgraph tailored to the question’s need.

Regarding the local knowledge base as a graph, G-RAG approaches generally retrieve a high-quality knowledge subset by fetching a subgraph that is both relevant to the query (question) and internally densely connected. With subgraph retrieval as the core technical module [6], current G-RAG methodologies still encounter the significant challenge of **coarse-grained exploration of graph topology**. Despite the potent representational power and rich semantic topology inherent in graph-structured data [8], the current utilization of graph structures in G-RAG methods remains straightforward. For instance, Approaches such as ToG [9], MindMap [10], DALK [11], and LightRAG [12] predominantly leverage immediate adjacency relationships to retrieve direct neighbors, local pathways, or n-hop subgraphs. HippoRAG [5] and GNN-RAG [13] employ graph algorithms like PageRank [14] or Graph Neural Networks [15] to assess node importance or identify relevant paths. These approaches focus on low-order structural information confined to pairwise or path-level connectivity, failing to capture the higher-order structural information [16] inherent in graph data.

Microsoft’s GraphRAG [4] and ArchRAG [17] represent initial efforts to explore deeper graph topology by treating subgraphs as communities [18, 19]. They apply graph clustering techniques [20] to partition large-scale knowledge graphs into distinct communities *a priori* (in the offline index construction process), and query over such fixed (static) clusters online. Nevertheless, such static and pre-computed communities may not fit the diverse queries that users issue to LLM. For example, consider a manager asking to summarise the employee cooperation among the departments for a certain project. This requires information cutting across predefined community boundaries, when the employee entities are offline clustered according to the department membership. Therefore, **how to dynamically mine high-order subgraph/community patterns subject to the diverse user queries**, remains a pivotal unresolved challenge.

In response to these challenges, we propose DA-RAG (Dynamic Atttributed community search for RAG). To the best of our knowledge, our work is the first to introduce and adapt the concept of Attributed Community Search (ACS) from graph analytics to serve

the specific needs of RAG. Specifically, we reframe the subgraph retrieval task in G-RAG as an embedding-attributed community search problem. This paradigm shift, illustrated in Figure 1, enables DA-RAG to dynamically identify a community from the knowledge graph that is both structurally cohesive and semantically guided by the query. Furthermore, to realize cost-effective online retrieval, DA-RAG is equipped with a chunk-layer oriented graph index, which primarily mirrors the logical structure of source documents by treating text chunks as graph nodes. The index further grows another two graph layers, considering the similarity and inherent connections among entities, respectively. In this way, the subgraph retrieval flows from the chunk layers to the two grown fine-grained layers. To summarize, our key contributions are as follows:

- (1) We pioneer a new subgraph retrieval paradigm for RAG by formulating an Embedding-Attributed Community Search (EACS) problem, which adapts ACS to dynamically retrieve structurally cohesive and semantically relevant subgraphs.
- (2) We design an efficient, multi-granularity Chunk-layer Oriented Graph Index that eliminates expensive clustering, reducing indexing costs while supporting queries at various levels of detail.
- (3) We demonstrate through extensive experiments that DA-RAG significantly outperforms state-of-the-art baselines in both response quality and end-to-end efficiency (indexing and retrieval).

2 Overview of DA-RAG

As illustrated in Figure 2, DA-RAG operates in two stages: a one-time **Offline Indexing** phase (Figure 2(a)) and a dynamic **Online Retrieval** process (Figure 2(b)). At the heart of its online retrieval lies a new subgraph search paradigm, which we formulate as the **Embedding-Attributed Community Search (EACS)** problem (Figure 2(c)). This integrated framework enables the retrieval of contextually rich and structurally coherent subgraphs, effectively addressing key challenges in G-RAG.

In the offline phase, DA-RAG processes an input document corpus to engineer a graph structure, termed the Chunk-layer Oriented

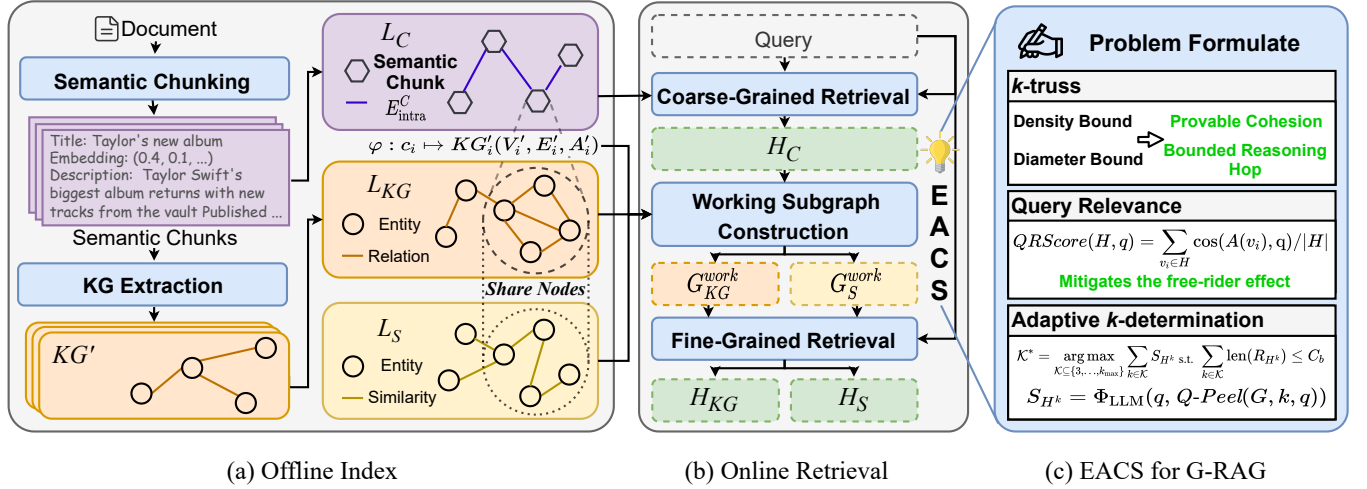


Figure 2: Overview of the DA-RAG framework: (a) Offline Indexing creates a novel graph index from source documents, comprising a high-level layer (L_C) and two granular layers (L_{KG} and L_S). (b) Online Retrieval employs a coarse-to-fine strategy. (c) EACS Formulation defines the subgraph retrieval in G-RAG as the Embedding-Attributed Community Search (EACS), ensuring provable cohesion, bounded reasoning hops, and mitigates free-rider effects.

Graph Index. Its cornerstone, the Semantic Chunk Layer (L_C), comprises nodes of semantic text chunks that provide high-level context by preserving the document’s narrative structure, thereby **avoiding expensive graph clustering** used in other methods [4, 17]. This layer is hierarchically linked to two fine-grained perspectives, the Knowledge Graph Layer (L_{KG}) and Similarity Layer (L_S) to form a dual-level, multi-perspective index, as detailed in Section 3.

During the online retrieval phase for queries, DA-RAG employs a **coarse-to-fine strategy**. First, a coarse search at the Chunk layer (L_C) identifies an initial community, H_C , which provides abstract, high-level context for the query [16]. This community then guides the pruning of the Knowledge Graph Layer (L_{KG}) and Similarity Layer (L_S), allowing for a final, fine-grained search within the resulting subgraphs. As a result, the process retrieves two detailed communities, H_{KG} and H_S , each offering different perspectives. For more details, please refer to Section 4.

Particularly, central to our online retrieval process is Embedding Attributed Community Search (EACS), detailed in Section 5, a novel query-guided subgraph retrieval paradigm, which guarantees **provable cohesion** and a **bounded reasoning hop** via k -truss, **mitigates the “free-rider effect”** [21] through a custom relevance score, and adaptive determination of k for k -truss.

3 Offline Index

The first stage of the RAG standard workflow involves organizing the knowledge base [2]. Our approach deeply leverages the inherent structure of standard G-RAG workflows [16]. We choose semantic chunking [22] over fixed-length methods [4]. This choice ensures that each chunk effectively captures a coherent segment of the document’s logic and narrative, yielding a set of semantic chunks denoted as $\{c_i\}_{i=1}^N$. We then conduct knowledge graph extraction [4] for each semantic chunk c_i to construct a local knowledge graph, denoted by the mapping $\varphi : c_i \mapsto KG'_i(V'_i, E'_i, A'_i)$.

Our core insight is that the combination of semantic chunking and knowledge graph extraction gives rise to an **emergent semantic hierarchy**. Each semantic chunk c_i serves as a high-level

abstraction over the detailed entities and relations in its corresponding graph KG'_i . This approach provides a cost-effective way to form a hierarchical structure, avoiding the need for computationally expensive methods like graph clustering.

To further enrich the index and mitigate the common issue of graph sparsity [23], we incorporate semantic similarity edges as proposed in previous work [11]. These edges are maintained in a separate Similarity Layer (L_S) to preserve the unique topology and relational semantics [24] of the Knowledge Graph Layer (L_{KG}). Summarizing the above insights, we propose a three-layer synergistic index as detailed in the following:

Semantic Chunk Layer (L_C): At a coarse-level, each node v_i^c in this layer represents a semantic text chunk c_i . We use an LLM to generate a title and a concise description for each chunk, which is further turned into a vector representation. Crucially, this embedding process utilizes the same embedding model employed to generate the query embedding, ensuring consistency in the vector space. Inspired by hierarchical clustering approaches [20], if the knowledge subgraphs extracted from two distinct text chunks, c_i and c_j ($i \neq j$), are connected by a relation in the global KG (i.e., there exists a relation $(u, w) \in E$ such that one entity is in V'_i and the other is in V'_j), we add an edge E_{intra}^c between their corresponding chunk nodes v_i^c and v_j^c in this layer as

$$E_{intra}^c = \{(v_i^c, v_j^c) \mid (i, j) \in P\},$$

$$P = \{(i, j) \mid i \neq j \text{ and } \exists u \in V'_i, w \in V'_j \text{ s.t. } (u, w) \in E\}.$$

Knowledge Graph Layer (L_{KG}): As a fine-grained layer, L_{KG} refers the global knowledge graph extracted from the corpus, primarily containing entity nodes (V), relations between them (E), and associated entity embeddings (A).

The above two layers are connected through inter-layer links based on the mapping φ . Specifically, each chunk node $v_i^c \in L_C$ is linked to all entity nodes $u \in V'_i$ within its corresponding knowledge subgraph.

$$E_{inter} = \{(v_i^c, u) \mid v_i^c \in L_C, u \in V'_i\}.$$

Similarity Layer (L_S): As another fine-grained layer complementary to L_{KG} , this layer comprises the same nodes (entities) as L_{KG} but with edges defined by semantic proximity. We employ the k -Nearest Neighbors (KNN) algorithm to build the Similarity Layer (L_S). In this layer, we connect each node to its top- k similar neighbors based on embedding similarity. A sensitivity analysis to identify the optimal value of $k_{neighbor}$ is provided in Section 6.3. Additionally, we create interlinks between this layer and the chunk layer, similar to our previous approach.

Overall, our offline process builds a dual-level (Semantic Chunk Layer L_C + Knowledge Graph Layer L_{KG}) and multi-perspective (structural relations in L_{KG} + semantic similarity in L_S) index.

4 Online Retrieval Workflow

Given the indexed corpus, we develop an efficient, coarse-to-fine retrieval strategy that narrows the search range when the queries arrive. Specifically, this strategy progressively reduces the search space by breaking down the overall retrieval task into a series of sub-retrieval steps. Each step is specified as a subgraph retrieval problem, aiming to identify the optimal subgraph from different graph layers. In this section, we will first outline the overall workflow of our proposed strategy.

Coarse-Grained Retrieval. Given the embedding q derived from the user’s natural language query, we initiate the retrieval process by performing EACS (a subgraph retriever to be detailed in Section 5) on the coarse level. We specifically operate at the Semantic Chunk Layer L_C since this approach is less computationally intensive. It identifies and generates an attribute community $H_C \subseteq L_C$, H_C providing a contextual anchor for subsequent fine-grained exploration.

Working Subgraph Construction. Leveraging the inter-layer connections E_{inter} , the retrieved chunk community H_C guides the identification of relevant entities within the Knowledge Graph Layer L_{KG} and Similarity Layer L_S . Specifically, we collect all entity nodes that are connected to any chunk node within H_C , forming the entity set $V_{work} = \{u \in V \mid \exists v_i^c \in H_C \text{ s.t. } (v_i^c, u) \in E_{inter}\}$. Based on this entity set, we induce two working subgraphs: G_{KG}^{work} for entities V_{work} within the layer L_{KG} , and G_S^{work} for them within the layer L_S .

Fine-Grained Retrieval. For a further refinement before generation, we execute EACS again on these significantly smaller working subgraphs G_{KG}^{work} and G_S^{work} . This step aims to identify fine-grained communities: H_{KG} within G_{KG}^{work} (representing relevant entities connected by explicit relations) and H_S within G_S^{work} (representing relevant entities connected by semantic similarity). These communities constitute fine-grained knowledge units highly relevant to the query and internally cohesive.

5 EACS: Key Module for Online Retrieval

The only thing we leave in the online stage is to decide the implementation of subgraph retrieval. Generally, subgraph retrieval of G-RAG can be represented as $G = \text{G-Retriever}(q, \mathcal{G})$, where G is a subgraph of the local database $\mathcal{G}$ and q is the user query. For this process, we observe that there exists a rational mapping from the area of G-RAG to attributed community search [25, 26], as shown in Figure 3. Specifically, the natural language query (q) provides semantic guidance, akin to the keywords and seed nodes in ACS. The

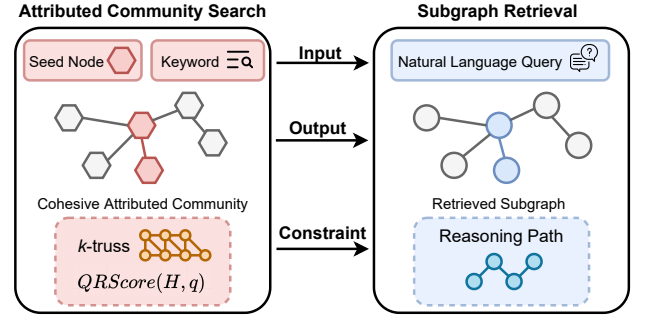


Figure 3: An illustration mapping the G-RAG subgraph retrieval task to the Attributed Community Search problem.

retrieved subgraph, our desired output, corresponds to the cohesive attributed community. Crucially, the implicit requirement for a “reasoning path” within the subgraph imposes a structural constraint that mirrors the cohesiveness metrics (e.g., k -truss) used in ACS to ensure the community is tightly connected [27]. We then novelly formulate the embedding-attributed community search problem as below, which is expected to capture the high-order semantic connections among nodes in the knowledge graph $\mathcal{G}$.

5.1 EACS Definition

Given a knowledge graph $G(V, E, A)$, natural language query q , a k -truss [28] parameter k (value will be determined at the end of this section), the problem of attributed community search returns a subgraph $H \subseteq G$ satisfying the following properties:

- (1) **Structure cohesiveness:** H is a connected k -truss;
- (2) **Query Relevance:** The query relevance score $QRScore(H, q)$ (Equation (1)) is maximized.
- (3) **Maximality:** There does not exist another subgraph $H' \supset H$ satisfying the above properties.

By design, these constraints provide the G-RAG pipeline with three critical advantages: **1) Provable Cohesion**, ensuring the retrieved context is thematically consistent; **2) Bounded Reasoning Hops**, which limits the reasoning space and prevents semantic drift; and **3) Mitigation of the Free-Rider Effect**, filtering out noisy or irrelevant nodes. In the subsequent analysis, we will deconstruct each of these constraints, demonstrating precisely the mechanism by which each confers its claimed advantage.

Constraint H is a k -truss. The k -truss constraint serves as the structural cornerstone for realizing two of our stated advantages: **Provable Cohesion** and **Bounded Reasoning Hops**. Its mechanism operates through two fundamental graph-theoretic properties. First, by definition, a k -truss requires every edge to be part of at least $k - 2$ triangles. This condition establishes a lower bound on density (we prove in Appendix A.1), which is the direct mechanism for Provable Cohesion, ensuring that the retrieved context is composed of thematically related concepts. Second, a connected k -truss has a guaranteed upper bound on its diameter, specifically $\lfloor \frac{2|V|-2}{k} \rfloor$ for a subgraph with $|V|$ vertices [28] (a concise proof is presented in Appendix A.2). This property directly translates into Bounded Reasoning Hops by imposing a finite upper limit on the path length between any two nodes. This restriction effectively curtails the reasoning space, preventing the semantic drift.

Constraint Query Relevance. The “free-rider effect” in graph analysis refers to the inclusion of nodes in a retrieved community primarily due to their structural connectivity [21], despite lacking direct relevance to the query’s core intent. Within G-RAG, such free-rider nodes can introduce noise and dilute the contextual information provided to the LLM, potentially degrading the quality and relevance of its generated responses [29]. Thus, we wish the retrieved community to avoid this effect. For this objective, we define a Query Relevance Score, which allows us to retrieve subgraphs that are closer to the query in the embedding space. Given a subgraph $H \subseteq G$ and an embedding function $f_{\text{embed}}(\cdot)$, the community semantic similarity of H , denoted as $QRScore(H, q)$, is the average similarity between nodes in the community and the query q :

$$QRScore(H, q) = \frac{\sum_{v_i \in H} \cos(A(v_i), f_{\text{embed}}(q))}{|H|}. \quad (1)$$

The EACS formulation, by optimizing for $QRScore$ within a k -truss structure, inherently **mitigates the free-rider effect**, as proved in Appendix A.3.

Adaptive k -determination The parameter k of k -truss for EACS controls the density and extent of the communities identified during both retrieval steps. Recognizing that the optimal k is inherently query-dependent, our DA-RAG framework infers the optimal k per query as described below.

We first generate a set of candidate communities, $\{H^k\}_{k=3}^{k_{\max}}$, by varying k over a feasible range. Subsequently, we leverage an LLM to perform a joint evaluation and summarization for each candidate. The process for each candidate community H^k is formally represented as $S_{H^k}, R_{H^k} = \Phi_{\text{LLM}}(q, H^k)$. Here, the LLM function Φ_{LLM} jointly generates two outputs based on the user query q and candidate community H^k : a relevance score S_{H^k} and a community report R_{H^k} . Finally, an optimal subset of the community is selected to maximize total relevance while adhering to the context budget C_b . The optimal set of community indices, $\mathcal{K}^*$, is determined by:

$$\mathcal{K}^* = \arg \max_{\mathcal{K} \subseteq \{3, \dots, k_{\max}\}} \sum_{k \in \mathcal{K}} S_{H^k} \quad \text{s.t.} \quad \sum_{k \in \mathcal{K}} \text{len}(R_{H^k}) \leq C_b.$$

We solve this via a greedy strategy: communities are ranked by their scores S_{H^k} and packed into the context sequentially until the budget is exhausted. This ensures the most relevant information is prioritized for final answer generation.

5.2 EACS Solution

We prove that EACS is an NP-hard problem (see Appendix A.4 for a detailed proof). Regarding such hardness, we propose an efficient heuristic named **Q-Peel**, an efficient multi-stage peeling algorithm to solve the EACS problem. The algorithm operates in three main phases. First, it prunes the input graph by extracting the maximal k -truss subgraph using a standard decomposition algorithm [30]. Second, it refines each connected component of this k -truss via an iterative peeling process. Specifically, nodes within a component are sorted in ascending order of their relevance to the query q . The algorithm then attempts to sequentially remove nodes, starting from the least relevant. A node is removed if it meets two conditions: it keeps the connected k -truss structure intact and improves the $QRScore$ from Equation (1). After processing all components, Q-Peel returns the subgraph with the highest $QRScore$. The overall

Algorithm 1 Q-Peel (Query-aware Peeling)

Require: G : Undirected attributed graph, q : Query embedding, k : Target k for k -truss

Ensure: H : Optimal community

```

1:  $T_k \leftarrow$  maximal  $k$ -truss subgraph of  $G$ 
2:  $C \leftarrow$  connected components in  $T_k$ ;  $updated \leftarrow \text{true}$ 
3: for all component  $c$  in  $C$  do  $S \leftarrow$  nodes in  $c$ 
4:   while  $updated$  do  $updated \leftarrow \text{false}$ 
5:     for all  $v$  in  $\text{SortByRelevance}(S, q)$  do
6:        $S' \leftarrow S \setminus \{v\}$ 
7:       if  $\text{IsValidImprovement}(S', S, q)$  then
8:          $S \leftarrow S'$ ;  $updated \leftarrow \text{true}$ 
9:       break
10:  if  $QRScore(H', q) < QRScore(S, q)$  then
11:     $H' \leftarrow S$ 
12: return  $H \leftarrow H'$ 
```

Q-Peel is illustrated in Algorithm 1, which shares a worst-case time complexity of $O(m^{1.5} + cn^2t)$ and a space complexity of $O(n + m)$, where n and m are the number of nodes and edges in the input graph, respectively. Detailed complexity proof can be found in Appendix A.5.

6 Experiments

In this section, we conduct a thorough experiment to evaluate the performance, answering the following research questions (RQs):

- **RQ1:** How does DA-RAG perform compared to baselines?
- **RQ2:** How efficient is the DA-RAG approach?
- **RQ3:** How is community quality retrieved by the DA-RAG method, and how does it affect RAG’s performance?

6.1 Experimental Settings

Datasets. Specifically, we utilize the **Agriculture** and **Mixed** subsets from the UltraDomain benchmark [31]. We also include the **News Articles** [32] dataset, previously employed in evaluating Microsoft’s GraphRAG [4]. We prompted the LLM to generate 125 challenging questions for each dataset following [12] for comprehensive evaluation.

Evaluation Metrics. We follow the studies [33] to try both position orders ($[R_a, R_b]$ and $[R_b, R_a]$) for each pair of evaluated RAG responses, and report the average win rate over all the questions and both position settings. There are four evaluation dimensions consistent with recent RAG studies [12, 16]: **Comprehensiveness**, **Diversity**, **Empowerment**, and **Overall**. Please see Appendix B.2 for more details on metrics.

Baselines. We evaluate our proposed method against several key baseline approaches from various representative G-RAG strategies. These include **LightRAG** [12], **HippoRAG** [5], **RAPTOR** [34], and community-partition-based methods such as **ArchRAG** [17] and **Microsoft’s GraphRAG** [4]. Additionally, we incorporate **VanillaRAG** [1] and **BM25** [35] as fundamental baselines. Our evaluation also considers the use of an LLM for answering questions without retrieval, specifically in **Zero-Shot** and **CoT** [36] contexts. All implementation details can be found in the Appendix B.

6.2 Main Results

Effectiveness (RQ1). We conducted head-to-head comparisons of DA-RAG against all baseline models across four evaluation dimensions for each dataset. The win rates are presented in Table 1, where

Table 1: Head-to-head win rates for our proposed DA-RAG versus baseline methods are reported as average percentages ($\pm$ standard deviation) over five experiments; results $\geq 50\%$ indicate that DA-RAG outperforms the baseline. In the table, GLightRAG, HLightRAG, and MLightRAG refer to three different variants of LightRAG, namely the Global, Hybrid, and Mix versions, respectively. Similarly, LGraphRAG and GGraphRAG correspond to the Local and Global variants of GraphRAG.

Win Rates of Comparison	Agriculture				Mix				News Articles			
	Comp.	Div.	Emp.	Over.	Comp.	Div.	Emp.	Over.	Comp.	Div.	Emp.	Over.
Inference-only												
DA-RAG vs Zero-shot	97.6(± 0.4)	95.7(± 0.5)	95.4(± 0.5)	92.8(± 0.7)	95.9(± 0.4)	92.8(± 1.0)	94.1(± 0.4)	94.7(± 0.5)	96.7(± 0.6)	95.5(± 0.3)	97.9(± 0.3)	95.8(± 0.8)
DA-RAG vs CoT	90.9(± 1.2)	94.7(± 1.3)	91.5(± 1.1)	90.8(± 1.5)	89.4(± 1.4)	87.8(± 0.9)	90.3(± 1.0)	90.0(± 1.1)	91.6(± 1.0)	90.2(± 0.8)	91.0(± 1.1)	91.2(± 1.3)
Retrieval-only												
DA-RAG vs BM25	93.7(± 1.1)	90.9(± 0.8)	93.6(± 0.9)	90.2(± 1.0)	92.8(± 0.9)	91.7(± 1.0)	93.0(± 0.8)	93.3(± 0.9)	92.1(± 1.2)	91.8(± 1.0)	92.2(± 0.9)	92.7(± 1.1)
DA-RAG vs VanillaRAG	93.9(± 1.0)	89.3(± 1.8)	89.9(± 1.3)	91.2(± 2.1)	87.2(± 1.3)	90.0(± 1.1)	91.3(± 1.3)	91.7(± 1.7)	94.8(± 1.6)	90.4(± 1.2)	90.1(± 0.9)	91.6(± 2.1)
Graph-based RAG												
DA-RAG vs GLightRAG	90.8(± 0.7)	90.1(± 1.0)	91.4(± 1.0)	91.1(± 0.9)	85.5(± 0.4)	92.9(± 0.7)	93.1(± 0.4)	93.7(± 0.9)	94.4(± 0.7)	93.1(± 0.9)	93.2(± 0.4)	93.1(± 0.9)
DA-RAG vs HLightRAG	90.7(± 0.4)	89.7(± 0.9)	90.0(± 1.1)	89.5(± 1.7)	85.3(± 0.8)	88.9(± 0.8)	90.7(± 0.4)	90.6(± 0.4)	92.9(± 0.7)	88.7(± 0.7)	92.1(± 1.1)	91.1(± 0.4)
DA-RAG vs MLightRAG	90.0(± 1.6)	83.3(± 2.3)	87.1(± 2.4)	87.0(± 1.5)	85.7(± 0.4)	87.7(± 1.3)	90.1(± 1.7)	89.9(± 2.0)	91.6(± 0.4)	89.2(± 1.1)	92.1(± 0.9)	90.2(± 1.0)
DA-RAG vs RAPTOR	84.3(± 1.1)	77.2(± 2.2)	82.5(± 2.3)	81.2(± 1.0)	88.7(± 0.7)	75.5(± 1.9)	86.4(± 0.7)	87.3(± 0.7)	86.2(± 1.3)	80.8(± 0.8)	85.5(± 1.2)	84.3(± 0.6)
DA-RAG vs HippoRAG	82.2(± 2.7)	74.3(± 1.6)	77.5(± 2.2)	76.4(± 1.1)	89.4(± 1.2)	73.7(± 0.7)	82.3(± 1.9)	82.3(± 2.3)	82.2(± 1.5)	84.4(± 1.1)	88.2(± 1.3)	86.9(± 1.9)
DA-RAG vs LGraphRAG	70.9(± 1.3)	67.0(± 1.6)	69.2(± 1.6)	67.8(± 2.4)	87.6(± 1.7)	80.8(± 3.4)	84.9(± 1.2)	83.3(± 2.4)	77.0(± 1.6)	75.7(± 1.6)	72.3(± 1.0)	75.3(± 1.1)
DA-RAG vs GGraphRAG	57.8(± 2.9)	57.1(± 3.2)	59.1(± 3.9)	56.6(± 4.2)	60.7(± 3.4)	50.7(± 1.0)	57.7(± 3.8)	55.2(± 1.9)	59.9(± 1.5)	61.5(± 1.3)	63.5(± 2.1)	61.7(± 0.8)
DA-RAG vs ArchRAG	50.3(± 3.9)	59.5(± 2.7)	53.1(± 4.3)	55.7(± 2.9)	52.6(± 1.9)	53.9(± 2.1)	58.2(± 3.3)	52.1(± 3.6)	52.4(± 3.9)	58.7(± 2.4)	55.5(± 2.5)	56.9(± 2.6)

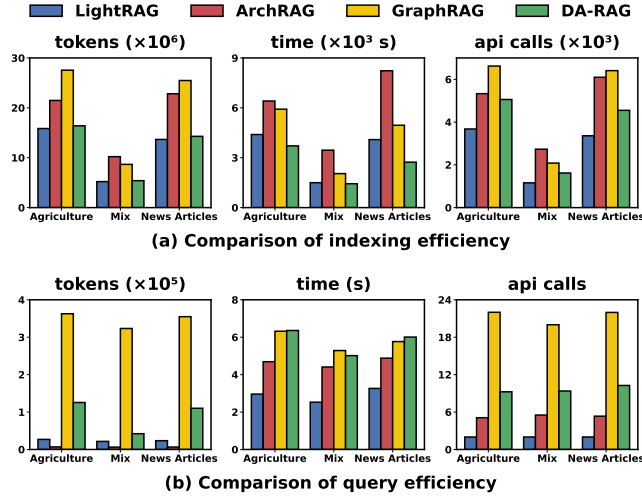


Figure 4: Efficiency comparison. DA-RAG is more efficient than ArchRAG and GraphRAG, while surpassing LightRAG in terms of effectiveness when given comparable efficiency. each row represents one baseline. DA-RAG consistently and significantly outperforms all baselines across all datasets and evaluation dimensions.

The complexity of the baseline models can stratify the analysis. First, the overwhelming win rates against non-retrieval methods (Zero-shot, CoT) and standard retrieval baselines (BM25, VanillaRAG) confirm the fundamental value of the RAG paradigm. For instance, the average overall win rate of 91.5% against VanillaRAG underscores the inherent limitations of simple dense vector retrieval and highlights the initial benefits of using structural information.

Second, when compared to Graph-based RAG methods that leverage low-order graph structures, confined to immediate neighbors

or connected paths, DA-RAG maintains a commanding lead. Notably, the substantial margins against the LightRAG variants (e.g., an average overall win rate of 92.6%) showcase the advantages of exploiting higher-order graph information (community).

Finally, the comparison with more advanced models, such as Microsoft’s GraphRAG and ArchRAG, is most revealing. As hypothesized, DA-RAG’s architecture demonstrates a clear advantage. The average win rates of 59.5% in Comprehensiveness and 60.1% in Empowerment against GGraphRAG validate the superiority of our dynamic, on-the-fly community identification over static, pre-partitioned graph communities. Our chunk-layer oriented graph index enables this query-specific context formation, leading to more thorough and relevant information synthesis.

Efficiency (RQ2). We assessed the efficiency of DA-RAG by measuring both its time cost (in seconds) and token usage (for LLM calls). As illustrated in Figure 4(a), the construction of the index reveals significant advantages for DA-RAG when compared to community-partition-based approaches. Peak reductions reach 37.3% in time and 41.8% in tokens with larger datasets like News Articles (when compared to GraphRAG). **DA-RAG consistently achieves optimal or near-optimal results** in terms of index construction efficiency. In online querying, as illustrated in Figure 4(b), DA-RAG maintains retrieval latency comparable to GraphRAG-Global while cutting total token consumption by an average of 73.8% (up to 88.76% on the Mix dataset). Specifically, our method imposes a total burden of only 9.3 API calls (42K tokens), including 8.3 calls (30K tokens) for the adaptive k -determination, which remains far below GraphRAG’s average of 21.3 calls (323K tokens). This significant efficiency gain is directly attributable to our proposed coarse-to-fine retrieval strategy.

To summarize, DA-RAG outperforms all the baselines while maintaining satisfactory running efficiency.

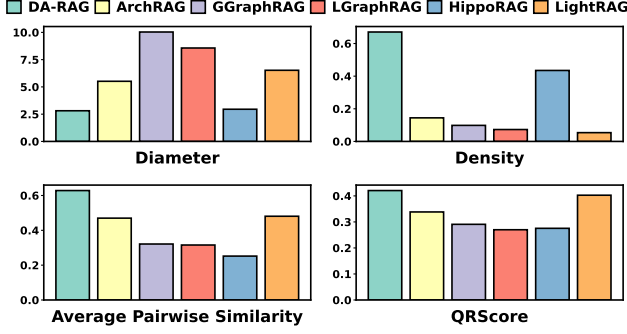


Figure 5: Analysis of retrieved subgraph quality on the Agriculture dataset. DA-RAG’s subgraphs exhibit the superior structural cohesiveness (highest density and lowest diameter) and semantic relevance (highest QRScore and similarity) compared to all baseline methods.

6.3 Further Analysis (RQ3)

Comparative Analysis of Subgraph Quality. To provide deeper insight into the effectiveness of our approach, we analyze the properties of the subgraphs retrieved by DA-RAG and baseline methods. As illustrated in Figure 5 for the Agriculture dataset, we assess four key metrics: diameter, density, QRScore and average pairwise similarity (detailed definitions can be found in the Appendix B.3). The results unequivocally demonstrate the superior quality of the subgraphs identified by DA-RAG.

Structurally, DA-RAG produces subgraphs with high density and low diameter. These properties are direct consequences of our EACS formulation, which leverages the k-truss constraint to ensure **Provable Cohesion** and **Bounded Reasoning Hops**. In contrast, low-order retrieval methods (e.g., HippoRAG, LightRAG) yield sparse and disconnected contexts. **Semantically, DA-RAG achieves the retrieval of both highly relevant to the query and internally coherent topics** as indicated by the highest QRScore and average pairwise node similarity. Moreover, while static community-based methods like GraphRAG can produce dense clusters, their lower semantic scores highlight a critical weakness: pre-computed partitions cannot adapt to the specific focus of a dynamic query.

In summary, it confirms that DA-RAG excels at retrieving context that is both semantically focused and structurally cohesive. This ability to construct a high-quality, compact knowledge subgraph for the LLM contributes to the significant performance gains observed in our main results (Section 6.2).

Case Study. We present a detailed case study in Table 2. The table contrasts the final contexts constructed by our DA-RAG framework and the GraphRAG baseline for the identical query: “How does celebrity endorsement shift consumer purchasing decisions?” The results reveal a stark difference in context quality. The context generated by DA-RAG is highly coherent and directly addresses the query. It successfully retrieves not only a real-world example, “Taylor Swift effect” on NFL merchandise sales, but also a **general principle of celebrity influence**. In contrast, the context produced by GraphRAG suffers from rigid subgraph partitioning, resulting in an overload of irrelevant information that does not relate

Table 2: Case study comparing context retrieved by DA-RAG and GraphRAG. The context is abridged for clarity.

Query: How does celebrity endorsement shift consumer purchasing decisions?

DA-RAG Context:

Taylor Swift effect: ...Following the publicization of their relationship... Kelce’s jersey sales soared by 400% ...

Celebrity Endorsements and Brand Visibility: Products benefit from celebrities who use them. ...This endorsement amplifies brand visibility and creates aspirational value ...

Engagement of Fanbase: The engagement of Swift’s extensive fanbase has drawn in millions, notably increasing ratings for games ...

GraphRAG Context:

Celebrity endorsements and their fallout: ...However, this association has since resulted in reputational damage for these individuals as FTX’s practices come under intense scrutiny ...

The role of Celebrity Endorsements: Celebrity Endorsements are strategically leveraged to influence Gen Z’s political perceptions ... The marketing approach by FTX underlined the importance of these endorsements. ...

to the query’s intent. This case vividly illustrates the core problem we identified in our introduction: the rigidity of the subgraph partition introduces substantial noise.

Ablation Study. To dissect the contributions of the core components of DA-RAG, we compare DA-RAG with the following variants:

- **DA-RAG w/o Similarity Layer (w/o L_S):** This variant removes the Similarity Layer (L_S) from the graph index.
- **DA-RAG w/o Semantic Chunk Layer (w/o L_C):** The EACS process is performed directly on the Knowledge Graph Layer (L_{KG}) and Similarity Layer (L_S).
- **DA-RAG w/o Semantic Chunking (w/o SC):** Replaces the semantic chunking method with a fixed-size chunking approach (1200 tokens/chunk with a 100-token overlap).
- **DA-RAG w/o ACS (using 1-hop Retrieval):** Replaces the EACS module with a widely adopted retrieval strategy [11].

Table 3(a) illustrates the individual contributions of key components in DA-RAG. **Our analysis shows that removing any single component leads to a noticeable decline in performance across all metrics.** In particular, excluding our dynamic attributed community search method (w/o ACS) or the Semantic Chunk Layer (w/o L_C) results in the most significant drops in performance. Specifically, DA-RAG w/o EACS shows win rates ranging from 25% to 41% compared to DA-RAG across four metrics. Similarly, DA-RAG w/o L_C experiences a decline in win rates from 50% to between 21% and 34%. These substantial decreases emphasize the critical role of EACS in retrieving relevant and cohesive subgraphs, while the L_C layer facilitates effective access to information at multiple granularities.

Varying LLMs. We evaluated our framework’s sensitivity to the choice of the LLMs, with results summarized in Table 3(b). This table presents the win rates of our DA-RAG method compared to Microsoft’s GraphRAG-Global baseline. **We found that the performance of DA-RAG improves when using more powerful integrated language models.** We posit that more advanced models like *gpt-4o* provide a richer and more accurate information

Table 3: (a) Win rates of DA-RAG variants vs. full model. (b) Sensitivity analysis on different LLM backbones.

Configuration	Comp.	Div.	Emp.	Over.
(a) Ablation Study				
w/o ACS	25.25%	30.56%	41.90%	33.33%
w/o L_C	21.43%	34.95%	25.53%	32.20%
w/o SC	40.21%	30.00%	31.96%	35.65%
w/o L_S	44.68%	40.78%	39.78%	40.52%
(b) Varying LLMs				
gpt-3.5-turbo	57.54%	59.57%	60.78%	58.36%
gpt-4o-mini	59.62%	61.74%	58.42%	58.12%
gpt-4o	60.43%	63.44%	62.38%	62.24%

substrate for our index. This enhancement amplifies the effectiveness of our DA-RAG framework by enabling adaptive community search to identify more coherent and semantically rich subgraphs tailored to specific queries, resulting in superior outcomes.

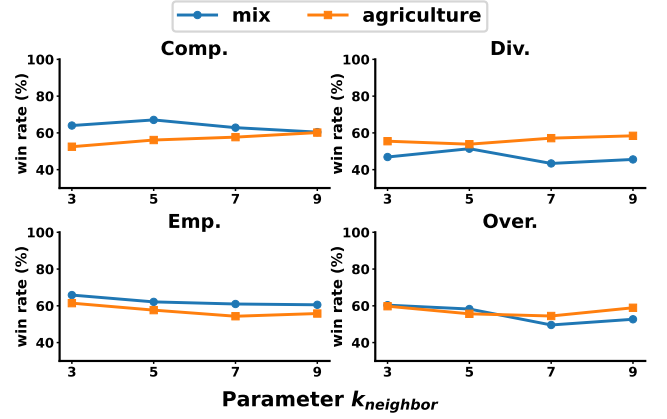
Hyperparameter Sensitivity Analysis We analyzed the sensitivity of our DA-RAG model to the hyperparameter k , the number of nearest neighbors used to construct the Similarity Layer (L_S). As shown in Figure 6, the model demonstrates strong robustness, with its win rates against Microsoft’s GraphRAG fluctuating only minimally as $k_{neighbor}$ varies from 3 to 9. We attribute this stability to our multi-perspective design, where the final context is synthesized from several layers (L_C , L_{KG} , and L_S). In this framework, the L_S layer acts as a complementary viewpoint by capturing essential node similarities, rather than being the sole driver of performance. Given that $k_{neighbor} = 5$ offers a consistent advantage, it is set as the default value in our experiments.

7 Related Work

Existing graph-based RAG approaches can be broadly categorized by how deeply they exploit structural information in graph data: **Adjacency Retrieval**, **Graph Topology Awareness**, and **High-order Pattern Mining**.

Adjacency Retrieval. Methods in this category [9–12, 37–41] utilize low-order structural signals, typically confined to immediate neighbors or connected paths. For example, LightRAG [12] retrieves top- k relevant entities and relations from the embedding space, followed by a one-hop expansion to create a subgraph as context. In addition, other works [9–11, 38, 39] employ Large Language Models (LLMs) to iteratively traverse graphs, exploring neighborhoods and constructing reasoning paths for inference. Nevertheless, all these methods are constrained by their local perspective, often failing to capture the broader, global associations present within the graph.

Graph Topology Awareness. To move beyond local adjacency, some research incorporates graph algorithms to capture the structural importance of nodes. HippoRAG [5, 42] applies PageRank [14] to assign global relevance scores to nodes given a query. Other works [13, 43–48] train Graph Neural Networks (GNNs) [15] to score node relevance, extracting top nodes and their connecting paths as context. Although these techniques enhance structural awareness, their focus typically remains on scoring individual nodes or simple paths. Consequently, they often overlook semantically

**Figure 6: Sensitivity Analysis on $k_{neighbor}$, where DA-RAG is considerably stable and $k_{neighbor} = 5$ offers the best results.**

richer, high-order patterns, such as communities or thematic clusters, which represent more abstract concepts.

High-order Pattern Mining. Recent work highlights the value of high-order structures in knowledge graphs, including communities, cliques, and other meaningful subgraph patterns. RAPTOR [34] organizes text into hierarchical trees via recursive clustering, capturing higher-level semantic groupings. Likewise, methods [4, 17, 49] detect community structures and produce summary reports for each, demonstrating notable gains in query-focused summarization tasks. However, most advanced methods [4, 17] rely on static, pre-computed structures that may not align with specific queries. To address this, we propose a framework that can dynamically discover structurally cohesive and semantically relevant subgraphs on-the-fly, tailored specifically to each query.

8 Conclusion

In this paper, we addressed the limitations of coarse-grained topological exploration in existing G-RAG methods, i.e., low-order structural information and static community partitions are inadequate for handling dynamic user queries. We introduced DA-RAG, which adaptively identifies and retrieves relevant knowledge subgraphs based on the semantics of the queries. Our experiments demonstrated the superior effectiveness of DA-RAG, achieving an average win rate of 57.34% over the GraphRAG-Global baseline, thus validating the benefits of dynamic retrieval. This work bridges graph analytics and retrieval-augmented generation, setting a new performance benchmark and establishing a novel methodological path.

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62472455, U22B2060), Key-Area Research and Development Program of Guangdong Province (2024B0101050005), Research Foundation of Science and Technology Plan Project of Guangzhou City (2023B01J0001, 2024B01W0004). Chen Zhang is supported by the NSFC/RGC Joint Research Scheme sponsored by the Research Grants Council of Hong Kong and the National Natural Science Foundation of China (Project No. N_PolyU5179/25); 2) the Research Grants Council of the Hong Kong Special Administrative Region, China (Project No. PolyU25600624); 3) the Innovation Technology Fund (Project No. ITS/052/23MX and PRP/009/22FX).

References

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Filippo Petroni, Vladimir Karpukhin, et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Neural Information Processing Systems, Neural Information Processing Systems* (May 2020).
- [2] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, et al. 2023. Retrieval-Augmented Generation for Large Language Models: A Survey. *CoRR* abs/2312.10997 (2023).
- [3] Haoyi Xiong, Jiang Bian, Yuchen Li, Xuhong Li, Mengnan Du, et al. 2024. When Search Engine Services Meet Large Language Models: Visions and Challenges. *IEEE Trans. Serv. Comput.* 17, 6 (2024), 4558–4577.
- [4] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, et al. 2024. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *CoRR* abs/2404.16130 (2024).
- [5] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. 2024. HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. In *NeurIPS*.
- [6] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, et al. 2024. Graph Retrieval-Augmented Generation: A Survey. *CoRR* abs/2408.08921 (2024).
- [7] Wenqi Fan, Yujian Ding, Liangbo Ning, Shijie Wang, Hengyun Li, et al. 2024. A Survey on RAG Meeting LLMs: Towards Retrieval-Augmented Large Language Models. In *KDD*. ACM, 6491–6501.
- [8] William L. Hamilton, Rex Ying, and Jure Leskovec. 2017. Representation Learning on Graphs: Methods and Applications. *IEEE Data Eng. Bull.* 40, 3 (2017), 52–74.
- [9] Jiashuo Sun, Chengjin Xu, Luminyuan Tang, Saizhuo Wang, Chen Lin, et al. 2024. Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. In *ICLR*. OpenReview.net.
- [10] Yilin Wen, Zifeng Wang, and Jimeng Sun. 2024. MindMap: Knowledge Graph Prompting Sparks Graph of Thoughts in Large Language Models. In *ACL (1)*. Association for Computational Linguistics, 10370–10388.
- [11] Dawei Li, Shu Yang, Zhen Tan, Jae Young Baik, Sukwon Yun, et al. 2024. DALK: Dynamic Co-Augmentation of LLMs and KG to answer Alzheimer’s Disease Questions with Scientific Literature. In *EMNLP (Findings)*. Association for Computational Linguistics, 2187–2205.
- [12] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2024. LightRAG: Simple and Fast Retrieval-Augmented Generation. *CoRR* abs/2410.05779 (2024).
- [13] Costas Mavromatis and George Karypis. 2025. GNN-RAG: Graph Neural Retrieval for Efficient Large Language Model Reasoning on Knowledge Graphs. In *ACL (Findings)*. Association for Computational Linguistics, 16682–16699.
- [14] Lawrence Page, Sergey Brin, Rajeev Motwani, and Terry Winograd. 1999. *The PageRank citation ranking: Bringing order to the web*. Technical Report. Stanford infolab.
- [15] Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Trans. Neural Networks* 20, 1 (2009), 61–80.
- [16] Yingli Zhou, Yaodong Su, Youran Sun, Shu Wang, Taotao Wang, et al. 2025. In-depth Analysis of Graph-based RAG in a Unified Framework. *Proc. VLDB Endow.* 18, 13 (2025), 5623–5637.
- [17] Shu Wang, Yixiang Fang, Yingli Zhou, Xilin Liu, and Yuchi Ma. 2025. ArchRAG: Attributed Community-based Hierarchical Retrieval-Augmented Generation. *CoRR* abs/2502.09891 (2025).
- [18] Mark Newman, Mark Newman, Michelle Girvan, and Michelle Girvan. 2004. Finding and evaluating community structure in networks. *Physical review E* 69, 2 (2004), 026113.
- [19] Santo Fortunato. 2010. Community detection in graphs. *Physics reports* 486, 3–5 (2010), 75–174.
- [20] Vincent A Traag, Ludo Waltman, and Nees Jan Van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific reports* 9, 1 (2019), 1–12.
- [21] Yubao Wu, Ruoming Jin, Jing Li, and Xiang Zhang. 2015. Robust Local Community Detection: On Free Rider Effect and Its Elimination. *Proc. VLDB Endow.* 8, 7 (2015), 798–809.
- [22] Umar Butler, Rob Kopel, Ben Brandt, and Jacobol. 2023. A fast, lightweight and easy-to-use Python library for splitting text into semantically meaningful chunks. <https://github.com/isaacus-dev/senchunk>.
- [23] Jay Pujara, Eriq Augustine, and Lise Getoor. 2017. Sparsity and Noise: Where Knowledge Graph Embeddings Fall Short. In *EMNLP*. Association for Computational Linguistics, 1751–1756.
- [24] Michael Sejr Schlichtkrull, Thomas N. Kipf, Peter Bloem, Rianne van den Berg, Ivan Titov, et al. 2018. Modeling Relational Data with Graph Convolutional Networks. In *ESWC (Lecture Notes in Computer Science)*, Vol. 10843. Springer, 593–607.
- [25] Yixiang Fang, Reynold Cheng, Siqiang Luo, and Jiafeng Hu. 2016. Effective Community Search for Large Attributed Graphs. *Proc. VLDB Endow.* 9, 12 (2016), 1233–1244.
- [26] Xin Huang and Laks V. S. Lakshmanan. 2016. Attribute Truss Community Search. *CoRR* abs/1609.00090 (2016).
- [27] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, et al. 2020. A survey of community search over big graphs. *VLDB J.* 29, 1 (2020), 353–392.
- [28] Jonathan Cohen. 2008. Trusses: Cohesive subgraphs for social network analysis. *National security agency technical report* 16, 3.1 (2008), 1–29.
- [29] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, et al. 2024. Lost in the Middle: How Language Models Use Long Contexts. *Trans. Assoc. Comput. Linguistics* 12 (2024), 157–173.
- [30] Jia Wang and James Cheng. 2012. Truss Decomposition in Massive Networks. *Proc. VLDB Endow.* 5, 9 (2012), 812–823.
- [31] Hongjin Qian, Peitian Zhang, Zheng Liu, Kelong Mao, and Zhicheng Dou. 2024. MemoRAG: Moving towards Next-Gen RAG Via Memory-Inspired Knowledge Discovery. *CoRR* abs/2409.05591 (2024).
- [32] Yixuan Tang and Yi Yang. 2024. MultiHop-RAG: Benchmarking Retrieval-Augmented Generation for Multi-Hop Queries. *CoRR* abs/2401.15391 (2024).
- [33] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, et al. 2024. Large Language Models are not Fair Evaluators. In *ACL (1)*. Association for Computational Linguistics, 9440–9450.
- [34] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, et al. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In *ICLR*. OpenReview.net.
- [35] Stephen E. Robertson, Steve Walker, Susan Jones, Micheline Hancock-Beaulieu, and Mike Gatford. 1994. Okapi at TREC-3. In *TREC (NIST Special Publication)*, Vol. 500-225. National Institute of Standards and Technology (NIST), 109–126.
- [36] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *NeurIPS*.
- [37] Yuntong Hu, Zhihan Lei, Zheng Zhang, Bo Pan, Chen Ling, et al. 2025. GRAG: Graph Retrieval-Augmented Generation. In *NAACL (Findings)*. Association for Computational Linguistics, 4145–4157.
- [38] Shengjie Ma, Chengjin Xu, Xuhui Jiang, Muzhi Li, Huaren Qu, et al. 2025. Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation. In *ICLR*. OpenReview.net.
- [39] Jing Zhang, Xiaokang Zhang, Jifan Yu, Jian Tang, Jie Tang, et al. 2022. Subgraph Retrieval Enhanced Model for Multi-hop Knowledge Base Question Answering. In *ACL (1)*. Association for Computational Linguistics, 5773–5784.
- [40] Antoine Bordes, Nicolas Usunier, Sumit Chopra, and Jason Weston. 2015. Large-scale Simple Question Answering with Memory Networks. *CoRR* abs/1506.02075 (2015).
- [41] Mufei Li, Siqi Miao, and Pan Li. 2025. Simple is Effective: The Roles of Graphs and Large Language Models in Knowledge-Graph-Based Retrieval-Augmented Generation. In *ICLR*. OpenReview.net.
- [42] Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. 2025. From RAG to Memory: Non-Parametric Continual Learning for Large Language Models. *CoRR* abs/2502.14802 (2025).
- [43] Zijian Li, Qingyan Guo, Jiawei Shao, Lei Song, Jiang Bian, et al. 2025. Graph Neural Network Enhanced Retrieval for Question Answering of Large Language Models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 6612–6633. <https://doi.org/10.18653/v1/2025.naacl-long.337>
- [44] Bill Yuchen Lin, Xinyue Chen, Jamin Chen, and Xiang Ren. 2019. KagNet: Knowledge-Aware Graph Networks for Commonsense Reasoning. In *EMNLP/IJCNLP (1)*. Association for Computational Linguistics, 2829–2839.
- [45] Guangyi Liu, Yongqi Zhang, Yong Li, and Quanming Yao. 2025. Dual Reasoning: A GNN-LLM Collaborative Framework for Knowledge Graph Question Answering. In *CPAL (Proceedings of Machine Learning Research)*, Vol. 280. PMLR, 351–372.
- [46] Xikun Zhang, Antoine Bosselut, Michihiro Yasunaga, Hongyu Ren, Percy Liang, et al. 2022. GreaseLM: Graph REASONing Enhanced Language Models. In *ICLR*. OpenReview.net.
- [47] Michihiro Yasunaga, Hongyu Ren, Antoine Bosselut, Percy Liang, and Jure Leskovec. 2021. QA-GNN: Reasoning with Language Models and Knowledge Graphs for Question Answering. In *NAACL-HLT*. Association for Computational Linguistics, 535–546.
- [48] Dhaval Taunk, Lakshya Khanna, Siri Venkata Pavan Kumar Kandru, Vasudeva Varma, Charu Sharma, et al. 2023. GrapeQA: GRaph Augmentation and Pruning to Enhance Question-Answering. In *WWW (Companion Volume)*. ACM, 1138–1144.
- [49] Rong-Ching Chang and Jiawei Zhang. 2024. CommunityKG-RAG: Leveraging Community Structures in Knowledge Graphs for Advanced Retrieval-Augmented Generation in Fact-Checking. *CoRR* abs/2408.08535 (2024).
- [50] Paul Burkhardt, Vance Faber, and David G. Harris. 2018. Bounds and algorithms for k-truss. *CoRR* abs/1806.05523 (2018).
- [51] Xiaxia Wang and Gong Cheng. 2024. A Survey on Extractive Knowledge Graph Summarization: Applications, Approaches, Evaluation, and Future Directions. In *IJCAI*. ijcai.org, 8290–8298.

A Problem Analysis

The k -truss [28] is an important cohesive subgraph concept extensively studied in the literature [26, 30, 50]. Burkhardt explored edge count ranges [50] for the connected k -truss, while Cohen established minimum diameter bounds [51]. In Appendices A.1 and A.2, we provide concise alternative proofs for the density and diameter bounds of the connected k -truss. Next, we formally define the free-rider effect for the G-RAG in Appendix A.3 and demonstrate that our proposed EACS problem effectively mitigates this effect. Lastly, Appendix A.4 contains complexity hardness proofs for EACS and an analysis of Algorithm 1 in Appendix A.5.

A.1 Lower Bound on Density of Connected k -truss

While Burkhardt [50] provided a range of edge counts for the connected k -truss, the density bound was not investigated further. In this part, we establish a density bound from a different perspective based on vertex degrees. Although the proof of the density bound is relatively trivial, to the best of our knowledge, it has not been formally provided previously.

Theorem. The density of a k -truss with $|V|$ vertices is greater than $\frac{k-1}{|V|-1}$.

Proof. First, recall the definition of a k -truss:

- In a k -truss, each edge must participate in at least $(k-2)$ triangles.
- This implies that the two endpoints of each edge must have at least $(k-2)$ common neighbors.

According to the degree sum formula in graph theory:

$$\sum_{v \in V} \deg(v) = 2|E|. \quad (2)$$

Since each vertex has degree at least $(k-1)$, we have:

$$2|E| = \sum_{v \in V} \deg(v) \geq |V|(k-1). \quad (3)$$

Therefore:

$$|E| \geq \frac{|V|(k-1)}{2}. \quad (4)$$

The density of a graph is defined as:

$$\text{density} = \frac{2|E|}{|V|(|V|-1)}. \quad (5)$$

Substituting the inequality from above:

$$\text{density} \geq \frac{|V|(k-1)}{|V|(|V|-1)} = \frac{k-1}{|V|-1}. \quad (6)$$

Thus, we have proven that the density of a k -truss is indeed greater than $\frac{k-1}{|V|-1}$.

A.2 Upper Bound on Diameter of Connected k -truss

This result has already been stated by Jonathan Cohen [28]. Here we present a novel proof approach based on Breadth-First Search (BFS) spanning tree.

Theorem. For any connected k -truss with $|V|$ vertices, its diameter d is at most $\lfloor \frac{2|V|-2}{k} \rfloor$.

Proof. Without loss of generality, select a vertex $r \in V$ and construct a breadth-first search (BFS) spanning tree T rooted at r

such that the height t of T is maximized. By definition, the diameter satisfies $d = t$.

We make two key observations regarding the BFS tree T :

- (1) Every edge of T corresponds to an edge in the original graph H .
- (2) No edge in H connects a vertex at level i in T to a vertex at level $i+2$ or higher, due to the BFS layer structure.

Consider an edge (u, v) in the BFS tree between L_i and L_{i+1} . Since H is a k -truss, the two endpoints of edge (u, v) must have at least $(k-2)$ common neighbors. The common neighbors must lie within levels i or $i+1$ due to observation 2. Therefore, for each $i = 0, 1, \dots, t-1$ the total number of vertices in the two consecutive layers $|L_i|, |L_{i+1}|$ satisfies

$$|L_i| + |L_{i+1}| \geq \underbrace{(1+1)}_{|u|+|v|} + (k-2) = k. \quad (7)$$

Summing this inequality over $i = 0, 1, \dots, t-1$ gives

$$\begin{aligned} \sum_{i=0}^{t-1} (|L_i| + |L_{i+1}|) &= (|L_0| + |L_t|) + 2 \sum_{i=1}^{t-1} |L_i| \\ &= 2|V(H)| - (|L_0| + |L_t|) \\ &\geq tk. \end{aligned} \quad (8)$$

Finally, since $|L_0| = 1$, and in the worst case minimizing total vertices we also take $|L_t| = 1$, we obtain

$$2|V| - 2 \geq tk \implies t \leq \frac{2|V| - 2}{k}. \quad (9)$$

As $d = t$ is an integer, the result follows:

$$d \leq \left\lfloor \frac{2|V| - 2}{k} \right\rfloor. \quad (10)$$

A.3 Mitigates the Free-rider Effect

Definition (Free-rider Effect for RAG). Given a query relevance score $f(q, \cdot)$ (the larger, the better). Let $H(V_h, E_h, A_h)$ be an optimal solution of the subgraph retrieval problem within the graph $G(V, E, A)$ for the query q . If there exists a node set $S \subseteq V$, where $S \not\subseteq V_h$, such that

$$f(q, G[S \cup V_h]) \geq f(q, H),$$

where $G[S \cup V_h]$ denotes the subgraph induced by node set $S \cup V_h$, then we say that the subgraph retrieval problem suffers from the free rider effect based on $f(q, \cdot)$.

Theorem. Let H be the discovered communities of the EACS with q . For any subgraph node set $S \subseteq V$, where $S \not\subseteq V_h$, it holds that

$$f(q, G[S \cup V_h]) < f(q, H).$$

Proof. We prove this statement by contradiction. Assume, for the sake of contradiction, that there exists a node set $S \subseteq V$, where $S \not\subseteq V_h$, such that $f(q, H \cup H^*) \geq f(q, H)$.

Since H is the solution to the EACS problem, it follows directly from its optimality that for any subgraph $H' \subseteq G$, we have

$$f(q, H') \leq f(q, H). \quad (11)$$

Combining this optimality property with our assumption, we have

$$f(q, H) \leq f(q, G[S \cup V_h]) \leq f(q, H). \quad (12)$$

By the squeeze theorem, we then have equality:

$$f(q, H) = f(q, G[S \cup V_h]). \quad (13)$$

Now, since $S \not\subseteq V_h$, it follows that

$$|G[S \cup V_h]| \geq |H|. \quad (14)$$

We consider two possible cases separately:

Case 1: If $G[S \cup V_h]$ is a connected k -truss, then $G[S \cup V_h]$ itself could serve as a candidate community satisfying the constraints of the EACS problem with q . $|G[S \cup V_h]| \geq |H|$. This contradicts the maximality assumption of the discovered community H , which is defined as an optimal solution to the EACS problem.

Case 2: If $G[S \cup V_h]$ is not a connected k -truss, then it does not satisfy the constraints required by the EACS problem. Hence, it cannot qualify as a feasible solution to the EACS problem.

In both cases, we arrive at a contradiction. Therefore, our initial assumption that $f(q, G[S \cup V_h]) \geq f(q, H)$ does not hold, and thus we must have

$$f(q, G[S \cup V_h]) < f(q, H). \quad (15)$$

This completes the proof. Therefore, we conclude that the EACS problem can avoid the free rider effect.

A.4 Hardness

In this section, we show the EACS is NP-hard. To this end, we define the decision version of the EACS problem and first prove its decision problem is NP-hard.

Problem (EACS-Decision). Given a attributed graph $G = (V, E, A)$, a natural language query q , parameter k_t , and threshold δ , the EACS-Decision problem is to determine whether there exists a connected subgraph $H \subseteq G$ that is a k_t -truss and satisfies $QRScore(H, q) \geq \delta$.

Theorem. The EACS-Decision problem is NP-Hard.

Proof. We prove this by a polynomial-time reduction from the well-known NP-hard problem *Maximum Clique* (decision version). Given an undirected graph $G' = (V', E')$ and an integer k' , the decision version of the Maximum Clique problem asks whether G' contains a clique of size k' .

Given such an instance (G', k') , we construct an instance of the EACS-Decision problem as follows:

- Let the attributed graph $G = (V, E, A)$ have the same graph structure as G' , i.e., $V = V'$, $E = E'$.
- For each node $v \in V$, assign the same attribute vector: $A_v = (1, 0, 0, \dots, 0)$.
- Define the query vector as $q = (1, 0, 0, \dots, 0)$.
- Set $k_t = k'$, and $\delta = 1$.

We now show that G' contains a clique of size k' if and only if the constructed EACS-Decision instance has a valid solution.

($\Rightarrow$) Suppose there exists a k' -clique C in G' . Let H be the induced subgraph of G on the nodes in C .

- H is connected since a clique is fully connected.
- In a k' -clique, every edge is part of exactly $k' - 2$ triangles. Thus, H is a k' -truss (i.e., every edge is in at least $k' - 2 = k_t - 2$ triangles).
- Each node in H has attribute vector $(1, 0, 0, \dots, 0)$, which matches the query vector, so $QRScore(H, q) = 1 \geq \delta$.

Therefore, H is a valid solution to the EACS-Decision problem.

($\Leftarrow$) Suppose there exists a subgraph $H \subseteq G$ that is a connected k_t -truss and satisfies $QRScore(H, q) \geq \delta$.

- Since $QRScore(H, q) \geq \delta = 1$, and all attribute vectors are $(1, 0, 0, \dots, 0)$.
- Since H is a k_t -truss, each edge in H participates in at least $k_t - 2 = k' - 2$ triangles.
- The only way to ensure this condition in general graphs (without creating additional structures) is to have H be a k' -clique, since in a k' -clique each edge is in exactly $k' - 2$ triangles.

Hence, H corresponds to a clique of size k' in G' .

The construction can be completed in polynomial time:

- Copying the graph structure takes $O(|V| + |E|)$ time.
- Assigning attribute vectors takes $O(|V|)$ time.
- Setting parameters takes $O(1)$ time.

Therefore, EACS-Decision is NP-hard.

A.5 Complexity Analysis

We now analyze the computational complexity of our proposed Algorithm, Q-Peel, with pseudocode in Algorithm 1, focusing on both time and space aspects.

Time Complexity. The algorithm consists of several key stages. In Line 1, it computes the maximal k -truss subgraph of G . This can be done in $O(m^{1.5})$ time, where m is the number of edges in the input graph G , using the standard k -truss decomposition algorithm [30]. Next, the algorithm iterates over each connected component of the k -truss subgraph and applies a node refinement process (Lines 3–10). Let c denote the number of such components, and $|S|$ denote the number of nodes in a component. During refinement, each node in S is considered for removal in each iteration of the while-loop. In the worst case, there can be up to $O(|S|)$ iterations, with one node removed per iteration. The subroutine `IsValidImprovement` can be performed in $O(t)$ time, where t is the number of edges in the 1-hop neighborhood subgraph of the removed node. Therefore, the total cost of processing one component is $O(|S|^2 t)$. In the worst case, where $|S| = O(n)$, the cost becomes $O(n^2 t)$. Considering all components, the worst-case time complexity of the algorithm is:

$$O(m^{1.5} + cn^2 t)$$

where c is the number of connected components in the k -truss, and t is the upper bound on the size of 1-hop neighborhood subgraphs. In practice, both c and t are typically much smaller than n , and the actual runtime is significantly reduced due to early termination of the refinement process.

Space Complexity. Let $n = |V|$ and $m = |E|$ be the number of nodes and edges in the input graph G , respectively. The algorithm maintains several auxiliary data structures:

- The k -truss subgraph T_k , stored as a subset of G , requires $O(n + m)$ space.
- The list of connected components C of T_k , requiring up to $O(n)$ space.
- For each component, a working node set S and its variants S' , consuming $O(n)$ space per component.
- Temporary structures used during refinement, such as priority queues for `SortByRelevance`, boolean flags, and candidate communities, each requiring at most $O(n)$ space.

Table 4: Detailed parameter configurations for the DA-RAG model.

Parameter	Value	Description
language_model_name	gpt-4o-mini	Language model used for response generation.
embedding_model_name	text-embedding-3-small	Model used to generate vector embeddings.
evaluation_model_name	gpt-4o-mini	Language model used for head-to-head comparison.
tiktoken_model_name	gpt-4o	Model used for token counting and encoding.
entity_extract_max_gleaning	1	Max iterations to refine entity extraction.
entity_summary_to_max_tokens	500	Max tokens allowed in the entity summary.
embedding_dimensions	1536	Dimensionality of embedding vectors.
embedding_max_token_size	8192	Maximum number of tokens that can be embedded at once.
embedding_func_max_async	16	Max number of asynchronous embedding calls.
language_model_max_async	16	Max number of asynchronous calls to the language model.
language_model_max_token_size	32768	Maximum context length supported by the language model.
key_string_value_json_storage	JsonKVStorage	Class used for JSON-based key-value storage.
vector_db_storage	NanoVectorDBStorage	Class managing vector database operations.
graph_storage	NetworkXStorage	Class for graph-based storage using NetworkX.
max_token_for_text_unit	4000	Token budget for single text unit.
max_token_for_context	4800	Token budget for retrieval context.
max_token_for_community_report	3200	Token budget for community report.

The most space-intensive operation is the maintenance of intermediate subgraphs during refinement. However, since these are all subgraphs of T_k , their cumulative space requirement remains bounded by $O(m)$. Therefore, the overall space complexity of the algorithm is: $O(n + m)$.

B Experimental Details

B.1 Implementation Details

Experiments were conducted on a Linux server equipped with an Intel Xeon 3.00 GHz CPU, 256 GB of RAM, and three NVIDIA GeForce RTX 3090 GPUs, each with 24 GB of VRAM. To reduce the randomness caused by the LLM, we set the response temperature to 0. For constructing the similarity layer within our chunk-layer oriented graph index, we employ k -Nearest Neighbors (KNN) to create edges between entities, with $k_{neighbor}$ set to 5. Detailed parameter configurations for the DA-RAG model can be found in Table 4. For baseline implementation, we applied the code provided in DIGIMON [16] for Hippo and RAPTOR. For GraphRAG, ArchRAG, and LightRAG, we utilized their officially released implementations.

B.2 Evaluation Metrics

To evaluate the quality of generated answers, we conduct a head-to-head comparison using an LLM-based evaluator. For this comparison, we adopt four metrics from previous work [12, 17], which are defined as follows. **Comprehensiveness**: How much detail does the answer provide to cover all aspects and details of the question? **Diversity**: How varied and rich is the answer in providing different perspectives and insights on the question? **Empowerment**: How well does the answer help the reader understand and make informed judgments about the topic? **Overall**: This dimension assesses the cumulative performance across the three preceding criteria to identify the best overall answer.

B.3 Definitions of Subgraph Property Metrics

This section provides the formal definitions for the metrics used to evaluate the retrieved subgraphs. Let Q denote the set of evaluation

queries. For each query $q \in Q$, G-RAG approaches may retrieve a set of subgraphs, denoted as $\mathcal{H}_q$. The values reported in the main paper represent the average of each metric computed over the entire collection of retrieved subgraphs from all queries:

$$\overline{\text{Metric}} = \frac{1}{\sum_{q' \in Q} |\mathcal{H}_{q'}|} \sum_{q \in Q} \sum_{H \in \mathcal{H}_q} \text{Metric}(H, q)$$

where $|\mathcal{H}_{q'}|$ is the number of subgraphs retrieved for query q' , and the total number of subgraphs is the denominator $\sum_{q' \in Q} |\mathcal{H}_{q'}|$. The term $\text{Metric}(H, q)$ represents the metric value for a specific subgraph H that was retrieved for query q . The calculation for a single subgraph $H = (V_H, E_H)$ is detailed below.

QRScore The QRScore quantifies the semantic alignment between a subgraph and the query that retrieved it. It's the same as Equation 1.

Density measures the internal structural cohesiveness of a subgraph H . It is the ratio of existing edges to the maximum possible number of edges for its set of nodes

$$\text{Density}(H) = |E_H| / (|V_H|(|V_H| - 1)).$$

Diameter reflects the structural compactness of the knowledge within H . It is defined based on shortest paths within the original global graph G . The diameter of a retrieved subgraph H is the maximum shortest path distance between any pair of its nodes that are reachable in G .

$$\text{Diameter}(H) = \max_{\substack{u, v \in V_H \\ d_G(u, v) < \infty}} d_G(u, v)$$

where $d_G(u, v)$ is the shortest path distance between u and v in G .

Average Pairwise Node Similarity assesses the internal semantic coherence of a subgraph H by averaging the cosine similarity over all unique node pairs within it.

$$\text{AvgSim}(H) = \frac{1}{\binom{|V_H|}{2}} \sum_{u, v \in V_H, u \neq v} \cos(A(u), A(v))$$